```
        kernel = getattr(program, kernel_name)
        kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
        self.queue.finish()
```

Используйте код с осторожностью.

Какие проблемы решает эта версия:

1. **Защита от дублирования аргументов ядра (`redefinition of parameter`)**: Аргументы собираются строго на основе уникального множества `detected_tensors_set`, предотвращая добавление одного и того же тензора (`gate_A`) дважды как указателя и как константы.
2. **Точечный инжект `[tid]`**: Теперь `[tid]` дописывается только к фактическим объектам `Tensor` на основе метаданных Python, исключая поломку вызовов функций (`nn_sigmoid(gate_vectors[tid])` отработает корректно).
3. **Строгая валидация по шагам**: Блок защищен от пропуска пустых строк, комментариев или ошибочных буквенных переменных, которые забыли объявить в Python-коде модели.

Если этот код логики прямого прохода вас устраивает, мы можем перейти к следующему шагу:

- Модифицировать метод `forward` в `test_autograd_integration.py`, заменив `self.runner(...)` на `self.runner.__call2__(...)` для проверки успешной компиляции ядра.

В целом предложенный тобой код меня устраивает , но в процессе реализации мне пришла в голову важная идея которую надо реализовать.
Мы заменим простое дописывание к тинзору значения [tid] на умный фильтр.
1. Извлекаем имена тензоров из стека.
2. К каждому имени (чистое имя , не содержит арифметических знаков, пробелов,символов «{[()]}» из множества имен тензоров , при применяем следующий алгоритм : в цикле обходим уникальные имена и добавляя к каждому имени знак из следующего массива « -+=, (/%» (пробел входит в это множество). Получаем вектор следующего вида: [`X `,`X=`,…]
На основе сформированного вектора имя+оператор, формируем вектор замен: [`X[tid] `,X[tid]=`…]
Для переданной строки выполняем алгоритм: пока значение вектора имя+оператор встречаеться в входной строке , меняем его на значенгие с tid из 2 вектора.

Это позволит компилировать без ошибок конструкции вида:
“X=Z[0]*Y”→ “X[tid]=Z[0]*Y[tid]”

Вот текущий код который нужно будет модефицировать.

```python
    def __call2__(self, program_text: str):
        # 1. Захватываем область видимости из фрейма вызова forward()
        caller_locals = sys._getframe(1).f_locals
        caller_globals = sys._getframe(1).f_globals
        self._captured_locals = caller_locals

        # Сбрасываем кэш захваченных тензоров
        self._captured_inputs.clear()
        self._captured_outputs.clear()

        # Разделяем входную строку по символу переноса "\n"
        raw_lines = program_text.strip().split('\n')
        compiled_statements = []
        detected_tensors_set = set()

        for line in raw_lines:
            cleaned_line = line.strip()
            if not cleaned_line or cleaned_line.startswith('#'):
                continue

            # Заменяем все "nnclloc." на "nn_"
            cleaned_line = cleaned_line.replace('nncclloc.', 'nn_')

            # Вплавляем значения скалярных параметров (например, q = 0.23 -> /=0.23)
            # Ищем потенциальные имена переменных в строке
            tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            for token in tokens:
                if token in ['nn_', 'sigmoid', 'tanh', 'dropout', 'tid']:
                    continue
                # Если токен найден в локальных переменных и это число (int, float)
                if token in caller_locals and isinstance(caller_locals[token], (int, float)):
                    val = caller_locals[token]
                    # Безопасно заменяем имя переменной на её числовое значение
                    cleaned_line = re.sub(r'\b' + token + r'\b', f"{val}", cleaned_line)

            # --- БЛОК ВАЛИДАЦИИ ИНСТРУКЦИИ ---

            # Шаг 1: Проверяем наличие хотя бы одного реального тензора в строке
            line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            has_tensor = False
            for t in line_tokens:
                if t in caller_locals and isinstance(caller_locals[t], Tensor):
                    has_tensor = True
                    detected_tensors_set.add(t)

            if not has_tensor:
                raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")

```

```
            # --- БЛОК ВАЛИДАЦИИ ИНСТРУКЦИИ ---

            # Шаг 1: Проверяем наличие хотя бы одного реального тензора в строке
            line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            has_tensor = False
            for t in line_tokens:
                if t in caller_locals and isinstance(caller_locals[t], Tensor):
                    has_tensor = True
                    detected_tensors_set.add(t)

            if not has_tensor:
                raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")

            target_tensor_name = None

            # Шаг 2: Проверяем наличие знака равенства "="
            if '=' not in cleaned_line:
                # Если знака равенства нет, должно быть строго ++ или --
                if '++' not in cleaned_line and '--' not in cleaned_line:
                    raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

                # Извлекаем имя тензора, к которому применен инкремент/декремент
                match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
                if match_inc_dec:
                    target_tensor_name = match_inc_dec.group(1)
            else:
                # Шаг 3: Разделяем строку по знаку равенства
                left_part, right_part = cleaned_line.split('=', 1)
                left_part = left_part.strip()
                right_part = right_part.strip()

                # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
                # Если перед '=' стоит один из знаков арифметики
                if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                    target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
                else:
                    # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                    target_tensor_name = left_part

                    # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                    right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                    for rt in right_tokens:
                        if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                            continue
                        # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                        if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                            continue
                        raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

            if not target_tensor_name:
                raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

            self._captured_outputs.add(target_tensor_name)
```

```
            if not target_tensor_name:
                raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

            self._captured_outputs.add(target_tensor_name)

            # Добавляем "[tid]" ко всем упоминаниям обнаруженных тензоров
            for t in detected_tensors_set:
                # Заменяем точечно по границам слов, избегая замен внутри системных имен
                cleaned_line = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cleaned_line)

            # Оборачиваем валидную строку в Си-конструкцию с проверкой границ
            compiled_statement = f"if (tid < {target_tensor_name}Size) {{ {cleaned_line}; }}"
            compiled_statements.append(compiled_statement)

        # Собираем входные тензоры-аргументы (все задействованные тензоры минус те, что перезаписаны)
        for t in detected_tensors_set:
            if t in caller_locals and caller_locals[t].grad_buffer is not None:
                if t not in self._captured_outputs:
                    self._captured_inputs.add(t)

        # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
        kernel_hash = abs(hash(program_text))
        kernel_name = f"text_forward_{kernel_hash}"

        # Формируем список аргументов ядра динамически на основе обнаруженных тензоров
        kernel_args = []
        cl_buffers_call_fwd = []
        max_elements = 1

        for tensor_name in sorted(list(detected_tensors_set)):
            tensor_obj = caller_locals[tensor_name]
            if tensor_obj.size > max_elements:
                max_elements = tensor_obj.size

            # Добавляем указатель на буфер данных и переменную размера
            kernel_args.append(f"__global float* {tensor_name}")
            kernel_args.append(f"const int {tensor_name}Size")
            cl_buffers_call_fwd.append(tensor_obj.data_buffer)
            cl_buffers_call_fwd.append(np.int32(tensor_obj.size))

        args_str = ", ".join(kernel_args)
        forward_statements_str = "\n    ".join(compiled_statements)

        # Собираем монолитный Си-код
        full_forward_code = f"""
{BASE_CL_FUNCTIONS}

__kernel void {kernel_name}({args_str}) {{
    int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
    {forward_statements_str}
}}
"""
        # Сборка сетки потоков и выполнение на GPU
        total_count = int(np.ceil(max_elements / float(self.warp_size)) * self.warp_size)
        global_size = (total_count // self.warp_size, self.warp_size)
```

0 ⚠ 0 Live Share